necessary to use dedicated annotations to match the business logic with the Spring Cloud Stream components. Now it’s completely transparent.

The framework supports integrations with RabbitMQ, Apache Kafka, Kafka Streams, and Amazon Kinesis. There are also integrations maintained by partners for Google PubSub, Solace PubSub+, Azure Event Hubs, and Apache RocketMQ.

This section will cover how to expose the composed function we defined in Dispatcher Service through message channels in RabbitMQ.

10.4.1 Configuring the integration with RabbitMQ

Spring Cloud Stream is based on a few essential concepts:

- *Destination binder*—The component providing the integration with external messaging systems, like RabbitMQ or Kafka
- *Destination binding*—The bridge between the external messaging system entities, like queues and topics, and the application-provided producers and consumers
- *Message*—The data structure used by the application producers and consumers to communicate with the destination binders, and therefore with the external messaging systems

All three of these are handled by the framework itself. The core of your application, the business logic, is not aware of the external messaging system. Destination binders are responsible for letting the application communicate with the external message brokers, including any vendor-specific concerns. The bindings are auto-configured by the framework, but you can still provide your own configuration to adapt them to your needs, as